

Project Codename: EchoNote AI

Vision

Build the world's best **fully offline AI meeting assistant** that captures conversations from a selected application (Chrome, Zoom, Teams, Slack, etc.) and/or microphone, performs real-time speech recognition, understands the conversation using local LLMs, and produces high-quality notes, Minutes of Meeting (MoM), action items, decisions, follow-ups, and searchable knowledge.

The application should feel comparable to products like Granola, Limitless, and Notion AI—but with one major differentiator:

100% Local. No cloud processing. No subscriptions. No user data leaves the machine.

Internet connectivity is required only for:

- Initial application installation
- AI model downloads
- Optional application updates

Everything else operates completely offline.

Product Principles

Local First

Every operation executes locally.

- Speech Recognition
- AI
- Embeddings
- Search
- OCR
- Export
- Encryption

No external APIs.

Privacy by Design

The application must guarantee:

- Audio never leaves the device.
- AI never leaves the device.
- Search index remains local.

- Notes remain local.
- Users explicitly choose whether audio recordings are stored.

Privacy is a core feature, not a setting.

Zero Configuration

The experience should be:

Install → Download models → Start recording.

No Docker. No Python. No command line. No configuration files.

Target Users

- Developers
 - Product Managers
 - Sales Teams
 - Consultants
 - Lawyers
 - Students
 - Researchers
 - Journalists
 - Doctors (where regulations permit)
 - Anyone attending online meetings
-

Core Features

Intelligent Audio Capture

Supported inputs:

- Microphone
- Selected application
- Microphone + selected application
- Multiple microphones
- Multiple applications (future)

Supported applications:

- Chrome
- Edge
- Firefox
- Safari
- Zoom

- Teams
 - Slack
 - Discord
 - VLC
 - Spotify
 - Any audio-producing application
-

Live Speech Recognition

Features:

- Live transcript
 - Near real-time updates
 - Punctuation restoration
 - Automatic language detection
 - Multi-language support
 - Timestamped transcript
 - Confidence scoring
-

AI Understanding

Generate:

- Meeting Summary
 - Minutes of Meeting
 - Action Items
 - Decisions
 - Risks
 - Open Questions
 - Follow-ups
 - Technical Summary
 - Business Summary
 - Customer Summary
 - Personal Notes
-

AI Workspace

Users can ask:

- "What decisions were made?"
- "Summarize only API discussions."
- "What tasks were assigned to me?"
- "Generate Jira tickets."
- "Write a follow-up email."
- "Convert to Markdown."
- "Create a project brief."

Everything is answered locally.

Session History

Each recording becomes a searchable knowledge object.

Store:

- Transcript
 - Notes
 - Summary
 - Audio (optional)
 - AI metadata
 - Speakers
 - Keywords
 - Embeddings
-

Universal Search

Search:

- Text
 - Semantic meaning
 - Speaker
 - Date
 - Topic
 - Action items
 - Decisions
-

Export

Export formats:

- Markdown
 - DOCX
 - PDF
 - HTML
 - JSON
 - Plain text
-

User Experience

Home

Shows:

Recent meetings

Search

Quick Record

Pinned sessions

Recent actions

Recording

Minimal UI:

Recording indicator

Audio levels

Selected application

Microphone

Live transcript

Live summary

Timer

Pause

Stop

Finished Session

Tabs:

Transcript

Summary

MoM

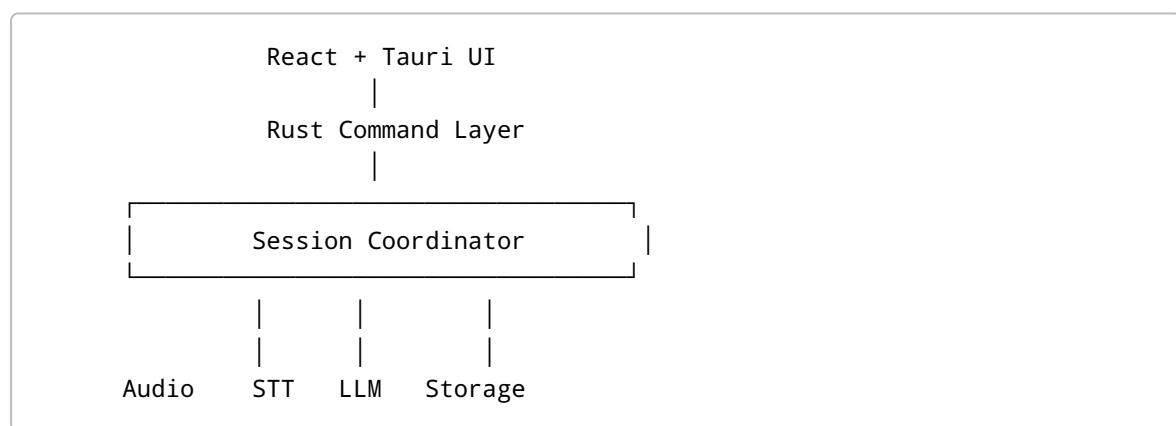
Actions

Decisions

AI Chat

Export

Technical Architecture



Rust Modules

```
src-tauri/  
  
audio/  
capture/  
transcription/  
llm/  
embeddings/  
search/  
session/  
database/  
models/  
settings/  
permissions/  
export/  
updates/  
telemetry/
```

Audio Engine

Responsibilities:

Application discovery

Microphone discovery

Permission handling

Audio routing

Mixing

Resampling

Echo reduction

Noise suppression

Voice Activity Detection

Timestamp synchronization

Speech Recognition Engine

Recommended Engine

whisper.cpp

Reason:

- Mature
- Fast
- GGML/GGUF ecosystem
- Metal
- CUDA
- Vulkan
- CPU
- Cross-platform
- Apache/MIT-friendly ecosystem

Future abstraction:

SpeechEngine

Whisper

NVIDIA Parakeet

Moonshine

Apple Speech

Local LLM

Instead of hardcoding one model, implement a provider layer.

LLM Provider

llama.cpp

MLX

ONNX Runtime

Future plugins

Recommended Default Models

Fast Summary

Qwen 3 4B Instruct GGUF

Excellent speed.

Excellent reasoning.

Runs comfortably on laptops.

Best Quality

Qwen 3 8B Instruct GGUF

Recommended default.

Very good structured outputs.

Excellent meeting understanding.

Apple Silicon Optimization

MLX versions of Qwen models.

Advantages:

Lower RAM

Lower latency

Higher throughput

Native Metal acceleration

Larger Models

Optional:

Llama 3.x 8B

Gemma 3

Mistral Small

DeepSeek distilled models

All installed through the model manager.

Embeddings

Recommended:

BGE Small

or

Nomic Embed

Used for:

Semantic search

Related meetings

Question answering

Database

SQLite

Encrypted using SQLCipher.

Stores:

Meetings

Transcript

Embeddings

Metadata

Settings

Search index

AI Pipeline

Audio

↓

Voice Activity Detection

↓

Streaming Whisper

↓

Transcript

↓

Rolling Context Builder

↓

Local LLM

↓

Summary

↓

Knowledge Graph

↓

Embeddings

↓

Search Index

Workflow

User presses Record

↓

Choose Application

↓

Choose Microphone

↓

Recording starts

↓

Transcript updates

↓

Rolling summary updates

↓

Action items detected

↓

Stop recording

↓

Final transcript refinement

↓

Final AI generation

↓

Embeddings generated

↓

Indexed

↓

Ready for search

Model Management

Built-in model manager.

Capabilities:

Browse models

Download

Pause

Resume

Delete

Update

Verify checksum

Storage location selection

No manual downloads.

Performance Goals

Initial launch:

<2 seconds

Recording latency:

<100 ms

Live transcript latency:

1-2 seconds

Summary refresh:

Every 20-30 seconds

Memory usage:

<3 GB during typical meetings (using default models)

CPU usage:

<30% on Apple M-series for normal meetings

Security

Encrypted local database

Encrypted recordings (optional)

No telemetry by default

Offline license-free operation

Secure model verification

Sandboxed permissions

Signed binaries

Automatic crash recovery

Roadmap

Phase 1

Core platform

Audio

Whisper

SQLite

Recording

Model manager

Phase 2

Summaries

MoM

Actions

Search

AI chat

Phase 3

Semantic search

Embeddings

Knowledge graph

Cross-meeting memory

Phase 4

OCR

Screen understanding

Slide extraction

Document generation

Jira/Git integration

Future Vision

EchoNote should evolve beyond a meeting recorder into a **personal local knowledge operating system**.

Every meeting, lecture, interview, voice memo, technical discussion, and brainstorming session becomes a searchable, AI-understandable knowledge asset—without relying on any cloud infrastructure.

The long-term goal is to provide an experience comparable to premium AI meeting assistants while maintaining complete user ownership of data, models, and workflows.